

Combinatorics

Circular Arrangements: n distinct objects in a circle: $(n-1)!$

Stars and Bars: Distribute n indistinguishable items into k distinguishable bins: $\binom{n+k-1}{k-1}$

Derangements: Permutations where no element is in its original position:

$$!n = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \frac{n!}{e}$$

Catalan Numbers: Paths on $n \times n$ grid not crossing above diagonal:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Spanning Trees (Cayley's): Distinct spanning trees in K_n : n^{n-2}

Cycle Counting: Distinct cycles of length k in complete graph K_n :

$$\binom{n}{k} \frac{(k-1)!}{2}$$

Probability & Expected Value

Linearity of Expectation (LoE): The expectation of a sum is the sum of expectations. Break complex counts into indicator variables $I_j \in \{0, 1\}$.

$$E[X] = \sum P(I_j = 1)$$

Law of Total Expectation: Condition on mutually exclusive outcomes Y to create a recursive equation:

$$E[X] = \sum_y E[X | Y = y] \cdot P(Y = y)$$

Coupon Collector's Problem: Expected trials to collect all n distinct items:

$$E[\text{Trials}] = n \sum_{k=1}^n \frac{1}{k} \approx n \ln(n) + \gamma n$$

Consecutive Successes (Resets): Expected attempts for n consecutive successes (failure completely resets):

$$E(\text{Attempts}) = \sum_{i=1}^n \frac{1}{P(\text{reach step } i \text{ consecutively})}$$

Bertrand's Ballot Theorem: Counting votes one by one.
Strict ($p > q$, no ties): Prob A strictly ahead: $\frac{p-q}{p+q}$
Weak (ties okay): Prob A never falls behind: $\frac{p+1-q}{p+1}$

Random Walks & Gambler's Ruin

Absorption Probability: Starting with i dollars, probability of reaching N dollars before hitting 0 (win prob p , loss $q = 1 - p$): If $p \neq q$:

$$P_i = \frac{1 - (q/p)^i}{1 - (q/p)^N}$$

If $p = q = 0.5$:

$$P_i = \frac{i}{N}$$

Expected Duration: Expected bets before hitting 0 or N in a fair game ($p = 0.5$):

$$E_i = i(N - i)$$

Continuous Prob. & Distances

Order Statistics (Uniform): If $X_1, \dots, X_n \sim U(0, 1)$ are independent, expected value of k -th smallest is:

$$E[X_{(k)}] = \frac{k}{n+1}$$

Expected Min: $\frac{1}{n+1}$ *Expected Max:* $\frac{n}{n+1}$

Geometric Probability (Meeting): Arrive independently uniformly between 0 and T , wait for time t . Prob of overlap:

$$P(\text{Meet}) = 1 - \left(1 - \frac{t}{T}\right)^2$$

Sum of Absolute Differences: For $i, j \in \{0, 1, \dots, N\}$:

$$\sum_{i=0}^N \sum_{j=0}^N |i - j| = \frac{N(N+1)(N+2)}{3}$$

Discrete Uniform Distance: For independent uniform random variables $X, Y \in \{0, 1, \dots, N\}$:

$$E|X - Y| = \frac{N(N+2)}{3(N+1)}$$

Useful Sums

Arithmetic & Geometric Series: $\sum_{k=1}^n k = \frac{n(n+1)}{2}$ $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$

$$\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1-r}, \quad \sum_{k=1}^{\infty} k r^{k-1} = \frac{1}{(1-r)^2} \quad (|r| < 1)$$

Algorithms / Code

Gaussian Elimination Example (Expected Value on Grid)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using vi = vector<ll>;

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    ll t; cin >> t; ll tc = 1;
    while(t--){
        ll n; cin >> n;
        vi jump(101,0);
        for(ll i=0; i<101; i++) jump[i] = i;
        for(ll i=0; i<n; i++){
            ll u,v; cin >> u >> v; jump[u] = v;
        }
        vector<vector<double>> mat(101,vector<double>(102,0.0));
        for(ll i=1; i<=100; i++){
            mat[i][i] = 1.0; mat[i][101] = 1.0;
            for(int roll=1; roll<=6; roll++){
```

```

        ll nxt = i+roll > 100 ? i : jump[i+roll];
        mat[i][nxt] -= 1.0/6.0;
    }
}
mat[100][100] = 1.0; mat[100][101] = 0.0;
for(ll col=1; col<=100; col++){
    ll pivotrow = col;
    for(ll i=col+1; i<=100; i++)
        if(abs(mat[i][col]) > abs(mat[pivotrow][col])) pivotrow = i;
    swap(mat[col],mat[pivotrow]);
    double pivotval = mat[col][col];
    for(ll j=col; j<=101; j++) mat[col][j] /= pivotval;
    for(ll i=1; i<=100; i++){
        if(i!=col){
            double factor = mat[i][col];
            for(ll j=col; j<=101; j++) mat[i][j] -= factor * mat[col][j];
        }
    }
}
cout << "Case " << tc++ << ": " << fixed <<
    setprecision(10) << mat[1][101] << "\n";
}
}

```

nCr Formula (Combinatorics with Modulo)

```

const int MAXN = 1e6 + 5;
const long long MOD = 1e9 + 7;
long long fact[MAXN], invFact[MAXN];

long long power(long long base, long long exp) {
    long long res = 1; base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD; exp /= 2;
    }
    return res;
}

void precompute() {
    fact[0] = 1; invFact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN - 1] = power(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 2; i >= 1; i--)
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

```